

PROJETO E ANÁLISE DE ALGORITMOS – 01A – 2022.2

[Notas de aula: solução de recorrências: método da substituição](#)

Uma recorrência é uma função definida recursivamente. Resolver uma recorrência significa fornecer uma expressão não recursiva para a função.

Recorrências aparecem na análise do tempo de execução de algoritmos recursivos. Portanto, precisamos resolver a recorrência para determinar a complexidade do algoritmo.

Exemplo: Potenciação recursiva (calcular a^n , com n é inteiro).

```
POT(a,n):
  se n = 0, retorne 1
  retorne a * POT(a,n-1)
```

Recorrência: $T(n) = T(n - 1) + \Theta(1)$, $T(0) = \Theta(1)$.

Exemplo: Números de Fibonacci.

```
FIBONACCI(n):
  se n = 0 ou n = 1, retorne 1
  retorne FIBONACCI(n-1) + FIBONACCI(n-2)
```

Recorrência: $T(n) = T(n - 1) + T(n - 2) + \Theta(1)$, $T(0) = T(1) = \Theta(1)$.

Exemplo: Ordenação Mergesort:

```
MERGESORT(A[], inicio, fim)
1 se inicio < fim                                1
2   meio <- piso( (inicio + fim) / 2 )           1
3   MERGESORT(A, inicio, meio)                  T(piso(n/2))
4   MERGESORT(A, meio+1, fim)                   T(teto(n/2))
5   INTERCALA(A, inicio, meio, fim)             Theta(n)
```

$$T(n) = T(\lfloor n/2 \rfloor) + T(\lceil n/2 \rceil) + \Theta(n), T(1) = \Theta(1).$$

Para simplificar as análises, assumimos que n é uma potência de 2, permitindo assim eliminar os pisos e tetos: $T(n) = 2T(n/2) + \Theta(n)$, $T(1) = \Theta(1)$.

Em cada caso, desejamos determinar $T(n) \in \Theta(?)$.

Solução de recorrências: método da substituição

Realize substituições até encontrar um padrão, e em seguida prove por indução que o padrão está correto.

Exemplo: Solução para $T(n) = T(n - 1) + 1$, $T(1) = 1$.

$$T(n - 1) = T(n - 2) + 1$$

$$T(n - 2) = T(n - 3) + 1$$

$$\begin{aligned} T(n) &= T(n - 1) + 1 \\ &= T(n - 2) + 1 + 1 \\ &= T(n - 3) + 1 + 1 + 1 \\ &= T(n - k) + k \\ &= T(1) + (n - 1) \\ &= n \end{aligned}$$

Provar por indução que $T(n) = n$.

Caso base: $n = 1$. $T(1) = 1$ (v).

Passo indutivo: assumo que vale para $T(n - 1)$, ou seja, $T(n - 1) = n - 1$.

Queremos provar que $T(n) = n$.

$$T(n) = T(n-1) + 1 = n - 1 + 1 = n.$$

Exemplo: Solução para $T(n) = T(n-1) + n$, $T(1) = 1$.

$$T(n-1) = T(n-2) + (n-1)$$

$$T(n-2) = T(n-3) + (n-2)$$

$$\begin{aligned} T(n) &= T(n-1) + n \\ &= T(n-2) + (n-1) + n \\ &= T(n-3) + (n-2) + (n-1) + n \\ &= T(n-k) + (n-k+1) + (n-k+2) + \cdots + n \\ &= T(1) + 2 + 3 + \cdots + n \\ &= 1 + 2 + \cdots + n \\ &= \frac{n(n+1)}{2} \end{aligned}$$

Provar por indução que $T(n) = n(n+1)/2$.

Caso base: $n = 1$. $T(1) = 1$, $1 \cdot (1+1)/2 = 1$ (v).

Passo indutivo: assumamos que vale para $T(n-1)$, ou seja, $T(n-1) = (n-1)n/2$.

Queremos provar que $T(n) = n(n+1)/2$.

$$T(n) = T(n-1) + n = (n-1)n/2 + n = n(n-1+2)/2 = n(n+1)/2.$$

Exemplo: $T(n) = 2T(n-1)$, $T(1) = 2$

$$T(n-1) = 2T(n-2),$$

$$T(n-2) = 2T(n-3)$$

$$T(n) = 2T(n-1) = 2(2T(n-2)) = 2(2(2T(n-3))) = 2^k T(n-k) = 2^{n-1} T(1) = 2^n.$$

Prova por indução que $T(n) = 2^n$.

Caso base: $T(1) = 2 = 2^1$.

Passo indutivo: assumamos que vale para $n-1$, ou seja, $T(n-1) = 2^{n-1}$. Então,

$$T(n) = 2T(n-1) = 2 \cdot 2^{n-1} = 2^n.$$

Generalizações

Teorema: $T(n) = aT(n-1) + f(n)$, $T(1) = f(1)$, com $a \geq 1$, tem solução $\sum_{i=1}^n a^{n-i} f(i) = \sum_{i=0}^{n-1} a^i f(n-i)$.

$$T(n-1) = aT(n-2) + f(n-1),$$

$$T(n-2) = aT(n-3) + f(n-2)$$

$$\begin{aligned} T(n) &= aT(n-1) + f(n) \\ &= a(aT(n-2) + f(n-1)) + f(n) \\ &= a(a(aT(n-3) + f(n-2)) + f(n-1)) + f(n) \\ &= a^3T(n-3) + a^2f(n-2) + af(n-1) + f(n) \\ &= a^kT(n-k) + a^{k-1}f(n-(k-1)) + \dots + a^0f(n) \\ &= a^{n-1}T(1) + a^{n-2}f(2) + \dots + a^0f(n) \\ &= a^{n-1}f(1) + a^{n-2}f(2) + \dots + a^0f(n) \\ &= \sum_{i=1}^n a^{n-i}f(i). \end{aligned}$$

Prova por indução que $T(n) = \sum_{i=1}^n a^{n-i}f(i)$.

Caso base ($n = 1$): $T(1) = a^{1-1}f(1) = f(1)$.

Passo indutivo: assumamos que vale para $n-1$, ou seja, $T(n-1) = \sum_{i=1}^{n-1} a^{(n-1)-i}f(i)$.

$$\begin{aligned} T(n) &= aT(n-1) + f(n) \\ &= a \sum_{i=1}^{n-1} a^{n-1-i}f(i) + f(n) \\ &= \sum_{i=1}^{n-1} a^{n-1-i+1}f(i) + a^0f(n) \\ &= \sum_{i=1}^{n-1} a^{n-i}f(i) + a^0f(n) \\ &= \sum_{i=1}^n a^{n-i}f(i). \end{aligned}$$

Exemplo (anterior): $T(n) = T(n-1) + 1$, $T(1) = 1$.

Neste caso $a = 1$ e $f(n) = 1$, portanto $T(n) = \sum_{i=1}^n a^{n-i}f(i) = \sum_{i=1}^n 1 = n$.

Exemplo (anterior): $T(n) = T(n-1) + n$, $T(1) = 1$.

Neste caso $a = 1$ e $f(n) = n$, portanto $T(n) = \sum_{i=1}^n a^{n-i}f(i) = \sum_{i=1}^n i = n(n+1)/2$.

Exemplo (anterior): $T(n) = 2T(n-1)$, $T(1) = 2$.

Neste caso $a = 2$, $f(1) = 2$ e $f(2) = \dots = f(n) = 0$, portanto $T(n) = \sum_{i=1}^n a^{n-i}f(i) = 2^{n-1}f(1) = 2^{n-1} \cdot 2 =$

Recursão de cauda

Na recursão de cauda removemos um elemento e resolvemos o subproblema resultante recursivamente. Além disso gastamos tempo $f(n)$ para preparar o subproblema e obter a solução final. Como exemplo temos o algoritmos PO

Corolário (recursão de cauda): $T(n) = T(n-1) + f(n)$, $T(1) = f(1)$, tem solução $\sum_{i=1}^n f(i)$.

Neste caso temos $a = 1$. Portanto, $T(n) = \sum_{i=1}^n a^{n-i}f(i) = \sum_{i=1}^n f(i)$.

Teorema (recursão de cauda polinomial): Se $T(n)$ é uma recursão de cauda e $f(n)$ é polinômio de grau $c \geq 0$, ou $T(n) = T(n-1) + f(n)$, $T(1) = f(1)$ e $f(n) = n^c$, então $T(n) \in \Theta(n^{c+1})$. Além disso, se $f(n) = n^c \log n$, então $T(n) \in \Theta(n^{c+1} \log n)$.

Ou seja, se $f(n)$ tem complexidade polinomial, então a recursão também terá (com incremento em 1 no grau).

Para provar usaremos aproximação de somatório através de integral. Como $f(n) = n^c$ é monótona não decrescente temos que

$$\begin{aligned} \int_{x=0}^n f(x)dx &\leq \sum_{i=1}^n f(i) \leq \int_{x=1}^{n+1} f(x)dx \\ \int_{x=0}^n x^c dx &\leq \sum_{i=1}^n i^c \leq \int_{x=1}^{n+1} x^c dx \\ \frac{n^{c+1}}{c+1} &\leq \sum_{i=1}^n i^c \leq \frac{(n+1)^{c+1} - 1}{c+1}. \end{aligned}$$

Como o limite inferior é $\Theta(n^{c+1})$, basta mostrar que o limite superior é $\Theta(n^{c+1})$:

$$\lim_{n \rightarrow \infty} \frac{(n+1)^{c+1}}{n^{c+1}} = \lim_{n \rightarrow \infty} \left(\frac{n+1}{n}\right)^{c+1} = \lim_{n \rightarrow \infty} \left(1 + \frac{1}{n}\right)^{c+1} = 1.$$

Da mesma forma, quando $f(n) = n^c \ln(n)$ podemos mostrar que $T(n) \in \Theta(n^{c+1} \log n)$:

$$\begin{aligned} \int_{x=0}^n f(x)dx &\leq \sum_{i=1}^n f(i) \leq \int_{x=1}^{n+1} f(x)dx \\ \int_{x=0}^n x^c \ln(x) dx &\leq \sum_{i=1}^n i^c \ln(i) \leq \int_{x=1}^{n+1} x^c \ln(x) dx \\ \frac{n^{c+1}((c+1)\ln(n) - 1)}{(c+1)^2} &\leq \sum_{i=1}^n i^c \ln(i) \leq \frac{(n+1)^{c+1}((c+1)\ln(n+1) - 1) + 1}{(c+1)^2}. \end{aligned}$$

Corolário: A recorrência $T(n) = a \cdot T(n-1) + c$, $T(1) = c$, com $a > 1$ e $c > 0$ constante, tem solução $T(n) \in \Theta$

Neste caso, $f(1) = f(2) = f(3) = \dots = f(n) = c$. Portanto,
 $T(n) = \sum_{i=0}^{n-1} a^i f(n-i) = \sum_{i=0}^{n-1} a^i c = c \sum_{i=0}^{n-1} a^i = c \left(\frac{a^n - 1}{a - 1}\right) \in \Theta(a^n)$.

Ou seja, quando $a > 1$ (mais de uma chamada recursiva) o algoritmo terá tempo exponencial, mesmo quando o c em cada chamada é constante!

Exemplos: $T(n) = T(n-1) + 1 \in \Theta(n)$, $T(n) = T(n-1) + n^2 \in \Theta(n^3)$, $T(n) = T(n-1) + n^2 \log n \in \Theta(n^3)$, $T(n) = 1.1T(n-1) + 1 \in \Theta(1.1^n)$.

[◀ Notas de aula: solução de recorrências: Teorema Mestre](#)

Seguir para...

[Notas de aula: solução de recorrências: árvore de recursão ▶](#)